

Platform Architecture: Universal OCR Intelligence

An engineering blueprint for an OCR aggregation platform and its recommendation agent — from request lifecycle to cold-start economics, engine onboarding, billing, and compliance.

Design principle

The agent (brain) decides; the execution layer (body) performs. Every architectural choice serves one of two goals: making the recommendation accurate, or making it cheap and fast to execute at scale.

00 Contents

01	Architecture at a Glance
02	System Layers & Request Lifecycle
03	The Recommendation Agent
04	Model Execution & Cold-Start Strategy
05	Engine Onboarding Pipeline
06	Billing, Metering & Multi-Tenancy
07	Security & Compliance
08	Recommended Tech Stack
09	MVP Scope & Build Sequence

01 Architecture at a Glance

The platform is a layered system: a thin client surface, a gateway that authenticates and meters, an orchestration core where the agent and router live, and an execution layer that runs engines across serverless GPU and commercial APIs — all supported by registry, billing, and observability services.

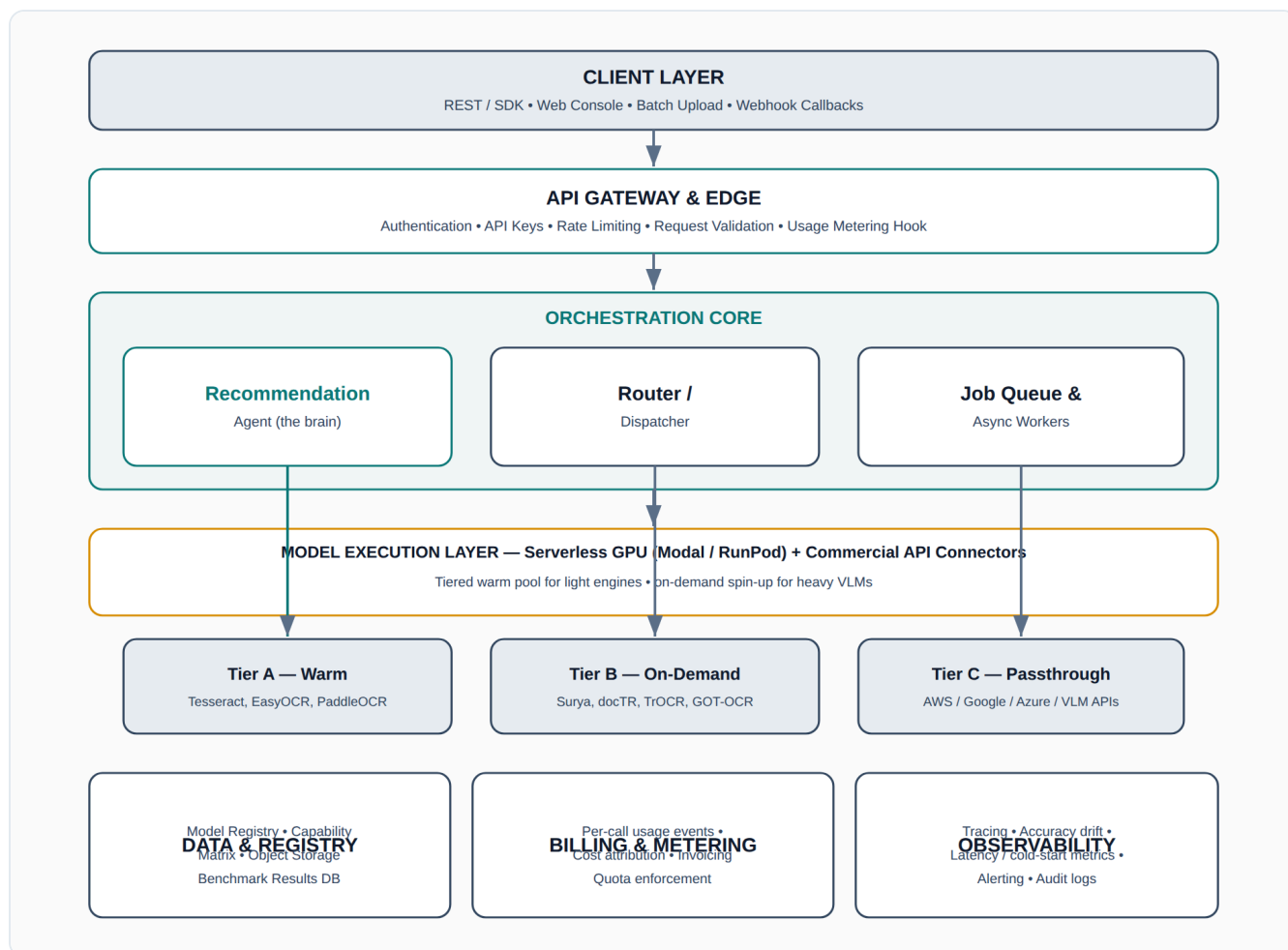


Figure 1 — End-to-end system architecture. The orchestration core is where differentiation lives; everything below it is commoditized infrastructure done well.

READING THE DIAGRAM

A request flows top to bottom: client → gateway (auth + meter) → orchestration core (agent recommends, router dispatches) → execution layer (the right engine tier runs) → result returns, with usage events emitted to billing and traces to observability throughout.

02 System Layers & Request Lifecycle

Layer	Responsibility	Key components
Client	How customers interact	REST API, language SDKs, web console, batch upload, webhook callbacks for async jobs
Gateway & Edge	Front door & control	Auth, API-key management, rate limiting, request validation, usage-metering hook
Orchestration Core	The decision-making brain	Recommendation agent, router/dispatcher, async job queue + workers
Execution	Running the engines	Serverless GPU pools (tiered), commercial API connectors
Supporting	State & operations	Model registry, object storage, benchmark DB, billing/metering, observability

Two request paths

Synchronous (single document, low latency)

Client → gateway validates & meters → router checks for a cached verdict for this document fingerprint → if known, routes directly to the chosen engine; if not, invokes the agent for a fast classification-based pick → engine runs → result + confidence returned. Target: keep within a tight SLA by favoring warm Tier-A engines and API fallback.

Asynchronous (batch / heavy VLM)

Client uploads batch → job enqueued, immediate job-ID returned → workers pull jobs, agent runs full trial-and-score where warranted, heavy engines spin up → results written to storage → webhook notifies client. This path absorbs cold-start latency where the customer expects throughput over instant response.

03 The Recommendation Agent

This is the product. Everything else is plumbing. The agent answers: *"For this specific document, which engine gives the best accuracy-per-dollar within the latency budget?"*

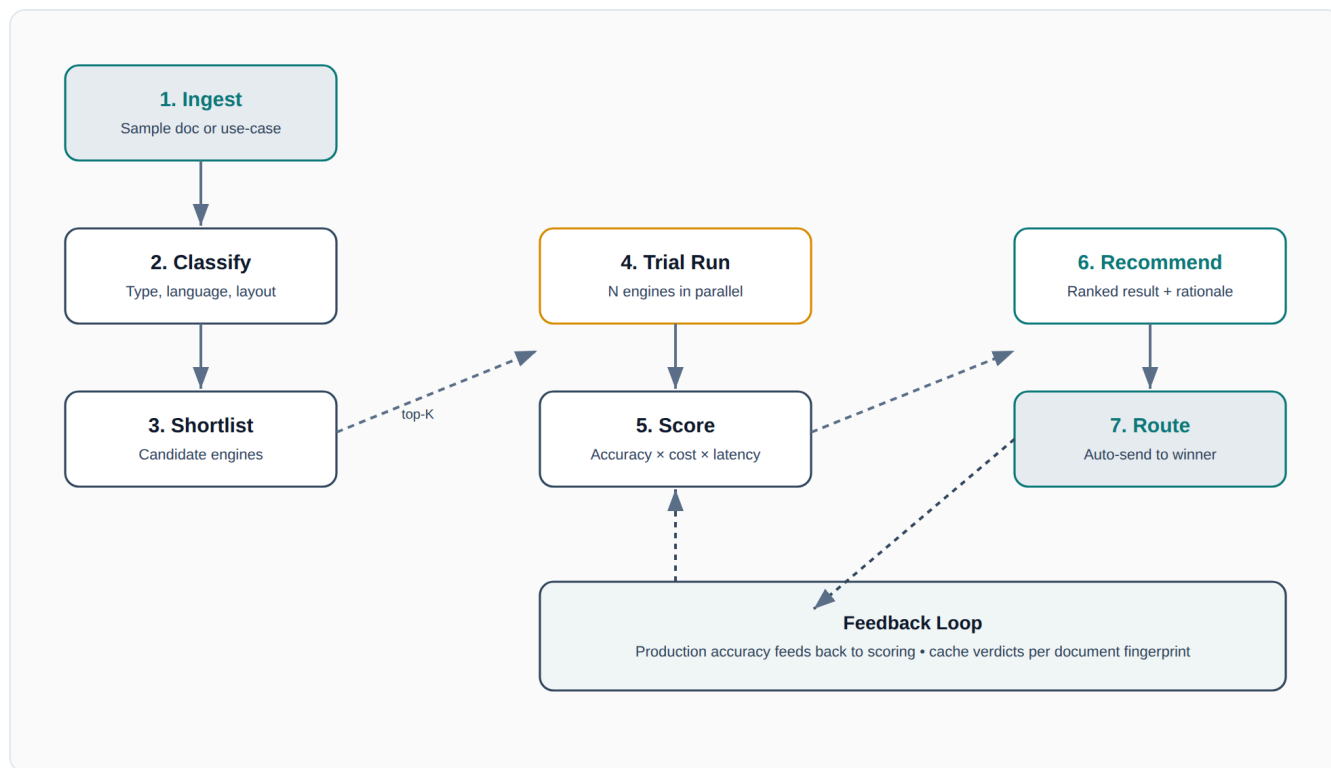


Figure 2 — The agent pipeline, from ingestion to live routing, with a feedback loop that compounds accuracy over time.

Stage by stage

- 1 Ingest.** Accept a sample document (image/PDF) or a structured use-case description (language, document type, volume, latency budget, cost ceiling).
- 2 Classify.** A lightweight vision model fingerprints the document: type (invoice, form, ID, prose, table-heavy), dominant language/script, layout complexity, print vs. handwriting, scan quality. This determines the candidate shortlist.
- 3 Shortlist.** Query the capability matrix in the model registry to select the top-K engines historically strong on this document profile — never run all engines, only the promising few.
- 4 Trial run.** Execute the shortlisted engines on the actual input in parallel (async path). This is the core differentiator: real evidence on real data, not generic leaderboard claims.
- 5 Score.** Combine accuracy (via consensus/ground-truth where available, or cross-engine agreement as a proxy), measured cost, and measured latency into a single ranked score weighted by the customer's stated priorities.
- 6 Recommend.** Return a ranked list with a plain-language rationale ("Engine X: best accuracy on your Urdu handwriting samples; Engine Y: 60% cheaper, 2% less accurate").

- 7 Route.** Auto-dispatch live traffic to the winner, with the verdict cached against the document fingerprint so repeat documents skip the trial.

THE COMPOUNDING MOAT

Production accuracy signals feed back into scoring, and cached verdicts per document-fingerprint mean the system gets faster and smarter with use. Over time the benchmark database itself — real engines vs. real documents — becomes proprietary IP no competitor can easily reproduce.

THE HARD PART, STATED PLAINLY

Scoring accuracy *without* ground truth is the genuine research challenge. Cross-engine agreement is a useful proxy but imperfect. Investment here — in consensus methods, lightweight human-in-the-loop validation, and per-vertical ground-truth sets — is what separates a real product from a wrapper.

04 Model Execution & Cold-Start Strategy

Serverless GPU makes the economics possible — you pay only when a request runs — but you also pay for the cold-start seconds before a model is ready. Managing this is the central infrastructure challenge.

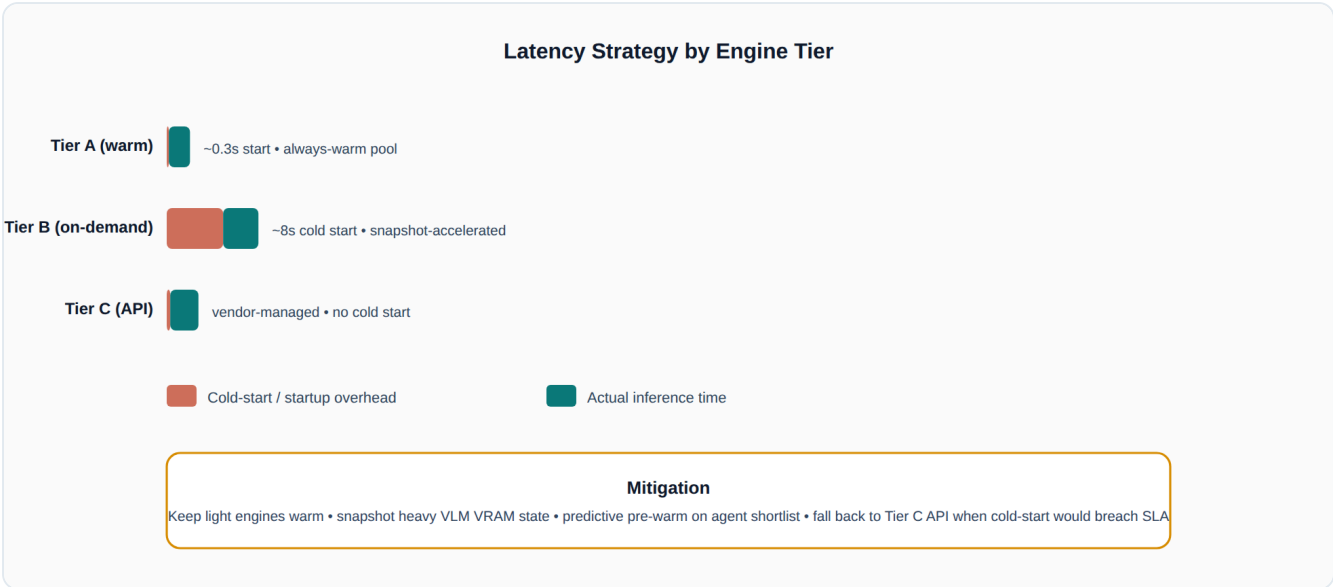


Figure 3 — Three engine tiers, three latency profiles. The strategy is to match each engine to the tier that fits its cost and responsiveness.

The three tiers

Tier	Engines	Hosting mode	Cold start
A — Warm	Lightweight: Tesseract, EasyOCR, PaddleOCR	Always-warm pool (cheap to keep resident)	~0.3s — effectively none
B — On-demand	Heavy VLMs: Surya, docTR, TrOCR, GOT-OCR, document VLMs	Serverless GPU, spun up per demand, VRAM-snapshot accelerated	~8s (snapshot) vs 30–60s naive
C — Passthrough	AWS / Google / Azure / hosted VLM APIs	Vendor-managed	None (vendor's problem)

Mitigation playbook

- **Keep light engines warm.** They are cheap to keep resident and cover a large share of everyday requests instantly.
- **Snapshot heavy VLM state.** Capturing model weights in VRAM cuts cold starts dramatically — the difference between a usable and unusable heavy engine.
- **Predictive pre-warm.** When the agent shortlists engines for a batch, pre-warm them before the trial run so the run itself isn't waiting.
- **SLA-aware fallback.** If a cold start would breach the customer's latency budget, the router transparently falls back to a Tier-C API for that request — correctness and responsiveness over ideological self-hosting.

COST REALITY

A ~10-second inference on an A100 costs on the order of a cent, but if 30 of those seconds are cold-start wait, the majority of your spend buys nothing. Tiering and snapshotting are not optimizations — they are what makes self-hosted margin exist at all.

05 Engine Onboarding Pipeline

"Every OCR on earth" is only sustainable if adding an engine is **repeatable, not bespoke**. This pipeline is the answer to the maintenance-burden concern.

- 1 **Standard adapter contract.** Every engine is wrapped to a common interface: `input(image/pdf, options) → output(text, layout, confidence, timing)` . Internal differences are hidden behind this contract.
- 2 **Containerize.** Package the engine + dependencies into a standard GPU/CPU container image with a defined handler, deployable to the serverless platform unchanged.
- 3 **Auto-benchmark on admission.** Run the new engine against the standard internal benchmark set (across document types, languages, layouts). Results populate the capability matrix automatically.
- 4 **Register capabilities.** The model registry records what the engine is good at, its cost profile, its tier assignment, and cold-start characteristics — the data the agent uses to shortlist.
- 5 **Canary & promote.** Route a small slice of suitable traffic to validate real-world behavior before full availability.

WHY THIS MATTERS STRATEGICALLY

Because onboarding is systematized, catalogue breadth scales with engineering effort linearly, not combinatorially. New open-source models — which now appear constantly — become a competitive advantage to add quickly, rather than a maintenance liability.

06 Billing, Metering & Multi-Tenancy

Precise metering protects margin

Every request emits a usage event capturing tenant, engine, tier, pages processed, compute-seconds (including cold-start time, tracked separately so it can be analyzed and optimized), and which revenue stream applies. Cost attribution at this granularity is what lets you price correctly and detect margin leaks per engine.

Concern	Approach
Usage events	Emitted at gateway and execution layer; aggregated asynchronously into the billing store
Cost attribution	Per-call compute-seconds × tier rate, plus passthrough vendor cost, mapped to the customer and stream
Quota enforcement	Checked at the gateway before dispatch; soft and hard limits per plan tier
Invoicing	Metered usage + subscription base; transparent line items by engine/stream
Multi-tenancy	Logical isolation per tenant; per-tenant keys, quotas, data partitions, and audit trails

07 Security & Compliance

Documents are among the most sensitive data a customer hands over — IDs, financials, medical records. Compliance is not a Phase-3 nicety; the foundations belong in the design from day one.

Area	Design response
Data residency	Region-pinned execution and storage; route requests to in-region engines for customers with residency requirements. (Mirrors why EU-native rivals win compliance-sensitive deals.)
PII handling	Encryption in transit and at rest; configurable retention with auto-purge; option for zero-retention processing where documents are never persisted.
The VLM ID-refusal trap	Some vision-language models <i>refuse</i> to process photo IDs or passports, returning a policy error mid-pipeline. The router must detect refusals and automatically reroute to an engine that will process the document — the agent's capability matrix flags which engines refuse which document classes.
Audit & access	Full audit logging of who accessed what; role-based access; per-tenant isolation enforced and logged.
Vendor data terms	Track and surface each passthrough vendor's data-use policy so customers know whether a given engine trains on or retains their data.

A SUBTLE PRODUCT RISK

The ID-refusal behavior is hard to predict per document and can break a pipeline silently. Detecting and rerouting around it is both a reliability feature and a genuine selling point — incumbents relying on a single VLM hit this wall.

08 Recommended Tech Stack

Layer	Recommendation	Rationale
API Gateway	Managed gateway (e.g. Kong / cloud-native) + lightweight auth service	Mature rate-limiting, key management, metering hooks
Orchestration	Python service (FastAPI) for the agent + router; async task queue (e.g. Celery/Temporal)	Python is the ML ecosystem's lingua franca; durable workflows for batch
Serverless GPU	Modal or RunPod Serverless (multi-provider abstraction recommended)	Per-second billing, scale-to-zero, snapshot support; avoid single-vendor lock-in
Agent classifier	Small, fast vision model for document fingerprinting	Cheap, runs hot, gates the expensive trial step
Model registry	Database + object store (capability matrix, weights, benchmark results)	Single source of truth the agent queries to shortlist
Storage	Object storage (S3-compatible), region-aware	Documents, batch results, residency control
Billing/metering	Event stream → aggregation store → billing engine	Granular, asynchronous, auditable
Observability	Tracing + metrics + alerting (latency, cold-start, accuracy drift)	Cold-start and drift are the metrics that protect margin and quality

09 MVP Scope & Build Sequence

The smallest thing that proves the thesis — that customers value the brain — with the least infrastructure.

MVP: the free recommendation tool + thin execution

In scope	Deliberately out of scope (for now)
Upload-a-document → ranked recommendation web tool	Full 100-engine catalogue
Document classifier + shortlist logic	Warm-pool guarantees / SLAs
5–8 curated engines (1–2 document VLMs, PaddleOCR, Tesseract, EasyOCR, 1–2 commercial passthroughs)	Enterprise compliance, on-prem, data residency
Trial-run + scoring on real input	Subscription tiers & team features
Basic API to route to the winning engine	Predictive pre-warm, advanced snapshotting
Simple metering for paid recommendation reports	Multi-region execution

Build sequence

- 1 **Engine adapters + containers** for the curated set, on one serverless GPU provider. Prove the execution contract.
- 2 **Document classifier** and capability matrix seeded from public benchmarks + your own runs.
- 3 **Trial-and-score loop** — the agent core. This is where to spend the most engineering care.
- 4 **Free web tool** wrapping the agent — the GTM wedge and validation instrument.
- 5 **Thin routing API + metering** so early adopters can move from recommendation to live usage and you can charge.
- 6 **Instrument everything** — latency, cold-start, cost-per-page, agreement scores — from the first day, because these numbers decide every later architectural trade-off.

THE SEQUENCING LOGIC

Build the brain against a small body first. Breadth (more engines), depth (compliance, SLAs), and margin engineering (snapshotting, pre-warm) are all Phase-2 concerns that only matter once customers confirm the recommendation itself is worth paying for.

This specification is a design blueprint, not a final engineering plan. Provider choices, latency figures, and cost estimates reflect 2026 research and should be re-validated against live provider pricing and benchmarks at build time.